

Práctica 2

Introducción a la electrónica con el LED RGB

Grado en Ingeniería en Robótica Software

GSyC, Universidad Rey Juan Carlos



©2024 Julio Vega Pérez

©2025 Esther Aguado

Algunos derechos reservados.

Este trabajo se entrega bajo licencia CC-BY-SA 4.0.

1. Introducción

La primera práctica fue sobre cómo manejar un LED (monocromo) mediante la técnica de PWM o modulación del ancho de pulso. Se puede considerar esta como el *Hola mundo* de la electrónica. Es fácil de instalar, se recuerdan conceptos de electricidad que ya tenías aprendido de antes y, además, el resultado es vistoso.

En esta ocasión, vamos a usar un LED más avanzado y, por tanto, más complejo de instalar y programar: el LED RGB. Evidentemente, las siglas RGB ya te resultarán familiares; son las siglas de *Red*, *Green*, *Blue*. Este *macroLED* contiene en una sola pieza esos tres LEDs¹ de colores. Además, y como ya sabrás, estos son los colores primarios de la luz, lo que significa que —modulándolos convenientemente— podrás conseguir cualquier color de luz.

Para manejar correctamente este LED, es importante comprender cómo funcionan los pines de entrada/-salida en Arduino.

2. Pines digitales

Como ya hemos visto en la Práctica 1, además de las conexiones de alimentación y comunicación, encontramos una serie de pines digitales y analógicos que se utilizan para interactuar con dispositivos electrónicos.

Ya hemos aprendido que los pines digitales pueden ser configurados como entrada o salida. Al configurarlos de salida, como hicimos en la Práctica 1, pueden ser activados o desactivados mediante código, es decir, ponerlo en HIGH (nivel alto, 5 V) o LOW (nivel bajo, 0 V). Del mismo modo, si se configuran como entrada, el pin puede leer HIGH o LOW dependiendo de la tensión leída.

Es importante tener en cuenta las limitaciones de corriente para evitar daños en la placa. Los pines de Arduino pueden suministrar de manera segura un máximo de 40 mA por pin, aunque el límite absoluto recomendado suele ser menor para proteger el dispositivo. Por ejemplo, al encender tres LEDs simultáneamente, se puede repartir la corriente máxima de forma aproximada de la siguiente manera:

$$I_{max} \approx 40mA/3 \approx 13mA \implies I_{max_R} = I_{max_G} = I_{max_B} = 13mA$$

¹Realmente, deberíamos hablar de *ledes* (igual que el plural de *red* es *redes*), aunque coloquialmente está más extendido el uso —incorrecto— de *LEDs*.

3. El LED RGB

Antes de conectar este LED, es conveniente analizar sus características eléctricas. Cada color del LED requiere un valor diferente de tensión y corriente: el rojo es el que menos tensión requiere, mientras que el azul es el que más tensión necesita.



Figura 1: LED RGB

Por otro lado, como vemos, este LED tiene cuatro pines en lugar de dos, como puedes comprobar en la Figura 1. Esto es debido a que, como ya hemos mencionado previamente, un LED RGB incluye en realidad un circuito de tres LEDs diferentes. Comúnmente, el pin más largo (posición 2) es el cátodo, mientras que los otros tres corresponden a los tres colores: rojo (1), verde (3) y azul (4).

Para salir de dudas y, como siempre, recuerda consultar la hoja de especificaciones de tu modelo concreto. Se adjunta la hoja de especificaciones de un LED RGB común. No obstante, el LED RGB que se incluye en el kit tiene la polaridad cambiada respecto a la norma común. Hay que tener esto en cuenta para la conexión del circuito (Sección 3.1).

3.1. Esquema de conexión

Para conectar el LED RGB y asegurarnos de no exceder las especificaciones de la placa, es necesario utilizar resistencias. Estas resistencias son fundamentales para proteger tanto los dispositivos conectados como, lo más importante, la placa.

Típicamente, los LEDs RGB soportan hasta 30 mA por pin, pero por seguridad los mantendremos por debajo de 13 mA, como habíamos calculado.

Las resistencias que tendremos que usar dependerán de las especificaciones concretas del LED RGB. En cualquier caso, obviamente necesitarás tres resistencias; una por cada pin de color. Para saber leer el código de colores de una resistencia, recuerda tener en cuenta lo siguiente:

1. Nosotros usaremos resistencias de cuatro bandas, aunque también las hay de cinco.
2. Aunque las resistencias son simétricas físicamente y se pueden conectar en cualquier sentido, para leer el código de colores correctamente hay que fijarse en la banda de tolerancia, que normalmente está más cerca de un extremo. Esa banda indica el final de la lectura, por lo que se debe empezar a leer desde el extremo opuesto.
3. Por si no estás totalmente seguro con el paso anterior, otra forma para asegurarnos de que estamos leyendo la resistencia correctamente es observar que existe un extremo en el que existe mayor separación entre la última banda y la penúltima; pues bien, este extremo ha de quedar a la derecha.

4. Ya teniendo bien situada la resistencia, observamos la primera y segunda banda. Estas nos dan el valor. Cada una va de 0 a 9, obteniendo por tanto un valor total comprendido entre 00 y 99.
5. La tercera banda es el multiplicador; esto es, por cuánto hemos de multiplicar el valor obtenido en el anterior paso. Así, por ejemplo, si el valor obtenido de las bandas 1 y 2 es 10 y el multiplicador es 100, el valor de la resistencia será de 1000Ω .
6. La cuarta banda es la tolerancia; esto es, el margen de error entre el valor dado según el código de colores y el valor real de la resistencia. Un margen de error o tolerancia habitual es el 5 %, representado por el color dorado.

También existen calculadoras *on-line*² que permiten obtener el valor de una resistencia a partir de sus bandas de colores.

Asimismo, se pueden utilizar calculadoras *on-line* específicas para resistencias de LEDs^{3,4}, que resultan muy útiles. Estas herramientas aplican directamente la conocida *Ley de Ohm*.

Para usar una de estas herramientas, simplemente introduce la tensión de la fuente, $V_S = 5\text{ V}$, la tensión que queremos que llegue al LED, por ejemplo $V_{LED} = 1,5\text{ V}$ (lógicamente, $V_{LED} \leq V_S$), y la corriente deseada para el LED (I_{LED}), recordando que $I_{LED} < 13\text{ mA}$. Con estos datos, la calculadora proporcionará el valor de la resistencia (R) necesaria, así como la potencia que disipará el LED (P).

Otra forma de trabajar, siguiendo la Ley de Ohm, consiste en *despejar* V_{LED} : es decir, indicar previamente la resistencia que vamos a usar, y en función de esta la herramienta nos mostrará la tensión que realmente llegará al LED.

Sea como sea, junto con las tres resistencias que finalmente insertemos y el cableado, resulta de especial importancia el uso de la *protoboard*. Con ayuda de esta, el esquema de conexión de un LED RGB común quedaría como vemos en la Figura 2.

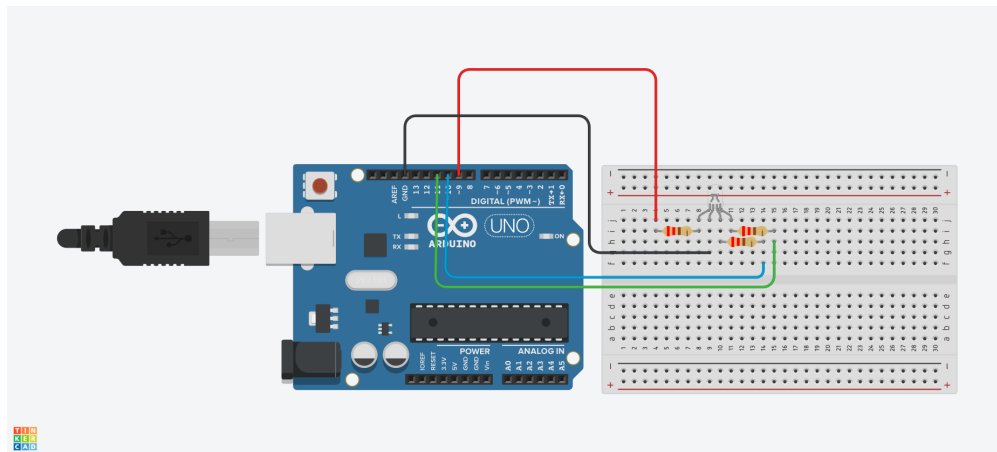


Figura 2: Esquema de conexión de un LED RGB común con protoboard.

Nótese que el esquema mostrado en la Figura 2 corresponde a la conexión típica de un LED RGB de cátodo común. En este caso, el pin común del LED se conecta a tierra (*GND*), mientras que cada uno de los tres ánodos se conecta a un pin programable distinto, en este ejemplo los pines 9, 10 y 11.

Sin embargo, el LED RGB incluido en el kit presenta la polaridad invertida respecto a esta configuración habitual. Esto significa que, en lugar de ser de cátodo común, se trata de un *LED de ánodo común*. Por ello,

²<https://www.digikey.es/es/resources/conversion-calculators/conversion-calculator-resistor-color-code>

³<https://ohmslawcalculator.com/led-resistor-calculator>

⁴<https://ledcalculator.net>

si se sigue literalmente el esquema de la Figura 2, el montaje no funcionará. La solución consiste en invertir las conexiones: la patilla larga debe considerarse como ánodo y conectarse a 5 V , mientras que los demás terminales se conectan a los pines de salida correspondientes a través de sus resistencias limitadoras. En este caso, su funcionamiento también se invierte en la programación: cada color se enciende cuando el pin recibe un nivel *LOW* (0) y se apaga cuando recibe un nivel *HIGH* (1).

Considérese esto una excepción, pues la mayoría de estos LEDs RGB suelen estar configurados como se ha explicado de forma general en los párrafos anteriores.

4. Programación del LED RGB

Una vez tenemos lo anterior, ya la programación es sencilla; basta con crearnos tres variables para guardar los pines correspondientes a cada uno de los tres colores y operar con ellos a nuestro antojo, como ya hicimos para el LED básico.

En el código que se facilita (`ledRGB.ino`) hemos definido tres macros para los pines del LED RGB:

```
1  const int redPin    = 9;
2  const int greenPin  = 11;
3  const int bluePin   = 10;
```

Se pueden crear funciones para encender o apagar un color concreto:

```
1  void encenderColor(int pin) {
2      digitalWrite(pin, HIGH);
3  }
4
5  void apagarColor(int pin) {
6      digitalWrite(pin, LOW);
7  }
8
9  void rojoON() {
10     encenderColor(redPin);
11 }
```

5. Ejercicios

Ejercicio 1. Uso de colores

1. Modifica convenientemente el programa para que pueda gestionar el encendido y apagado de los tres colores primarios.
2. Añade la funcionalidad necesaria para gestionar el encendido y apagado de, al menos, otros cinco colores, p. ej.: magenta, amarillo, cyan, blanco, etc.

Ejercicio 2. Interfaz de usuario

Partiendo de la funcionalidad implementada en el Ejercicio 1, utiliza el puerto serie de Arduino para solicitar información al usuario por teclado y controlar el LED RGB. Por ejemplo, el usuario puede enviar comandos como:

- `r1` para activar el rojo
- `g0` para apagar el verde
- `b1` para encender el azul

Comunicación Serial en Arduino

Arduino puede comunicarse con el ordenador mediante comunicación serial, lo que permite enviar y recibir datos a través del puerto USB. Esto es muy útil para interactuar con el usuario. En el IDE de Arduino se accede mediante el *Monitor Serial* (el icono de lupa situado arriba a la derecha).

Para habilitar la comunicación serial, se debe configurar en la función `setup()` usando:

```
1 Serial.begin(9600);
```

Este comando inicia la comunicación serial a la velocidad de transmisión especificada, expresada en bits por segundo (bps). Tanto Arduino como el Monitor Serial deben usar la misma velocidad para interpretar correctamente los datos. En nuestro ejemplo, usamos 9600 bps, un valor seguro y común para la mayoría de prácticas.

Además, Arduino dispone de otras funciones útiles para la comunicación serial.

- `Serial.print()` / `Serial.println()`: imprime los datos en formato texto, el segundo añade un salto de línea al final.
- `Serial.read()`: lee los datos enviados.

Para más información, puedes consultar la documentación oficial de Arduino sobre Serial.

```
1 void setup() {
2     Serial.begin(9600);
3     Serial.println("Introduce tu nombre");
4 }
5
6 void loop() {
7     if (Serial.available() > 0) {
8         String input = Serial.readStringUntil('\n');
9         Serial.print("Hola, ");
10        Serial.print(input);
11        Serial.println("!");
12    }
13 }
```

Entrega de la práctica

La entrega se realizará en el repositorio individual de GitHub Classroom. Cada repositorio debe contener al menos el código producido así como un archivo `README.md` que incluya la memoria de la práctica con los siguientes elementos:

La memoria de prácticas debe contener los siguientes elementos:

- Nombre de los estudiantes.
- Objetivo de la práctica.
- Resumen del trabajo realizado.
- Particularidades: Problemas encontrados, funcionalidad extra, etc.
- Vídeo del funcionamiento del ejercicio.
- Esquemático del circuito (Fritzing, Tinkercad, etc.).